

SRH University Heidelberg

Master's Thesis

Explainable AI for Image Classification: Revealing and Interpreting Model Memorization

Author: & Erdem, Zeynep Öykü
Matriculation Number: & 100002305
Address: & Emil-Henk-Straße 7
& 69115 Heidelberg
& Germany
Email Address: & zeynepoykuerdem@outlook.com
Supervisor: & Milan Gnjatovic
Begin: & 01. October 2025
End: & 01. April 2026

Abstract

burda ingilizce abstract

Abstract

burda almanca abstract olucak

Contents

0.1 Introduction 7
0.2 Literature Review and Background 9
0.2.1 Foundations of Image Classification and Deep Learning 9
0.2.2 The Taxonomy of Explainable AI in Image Classification 11
0.2.3 The Phenomenon of Memorization 17
0.3 Methodology 21
0.3.1 Mathematical Background of Neural Networks and Training 21
0.3.2 Mathematical Formalization of Memorization 25
0.3.3 Mathematics of White-Box Attacks (Adversarial Optimization) 26
0.3.4 XAI and Attribution Mathematics 27
0.3.5 Experimental Setup and Audit Pipeline 29
0.3.6 Evaluation Metrics and Analysis 30
0.4 Implementation and Structural Design 32
0.5 Evaluation and Results 33
0.6 Conclusion and Future Work 34
0.7 Appendix 35

List of Figures

1 The classification error comparison between pre-trained models with ImageNet dataset 10
2 The flowchart of taxonomy explanation 12
3 Different Interpretability Approach in Deep Learning Model 13
4 Adversarial Example Generated by FGSM 20
5 Graphical representation of the sigmoid activation function. 22
6 Graphical representation of the tanh activation function. 22
7 Graphical representation of the ReLU activation function. 23

List of Tables

0.1 Introduction

Over the past decade, Artificial Intelligence (AI) has been firmly established in our society widely adopted across various sectors by optimizing operations, supporting decision-making processes, and enhancing productivity through the reduction of operational inefficiencies [1] [AAES+23]. In healthcare, AI supports disease detection and diagnosis, and [2] [PF24] enables faster and more accurate medical interventions. In finance, AI is employed to detect fraud and manage customized financial services [3][OUM24], decide eligibility in loan approval. Law enforcement agencies utilize artificial intelligence to aid in decision making and crime prevention. [4] [SHZ+24], [5] [JMH24], while the automotive sector leverages AI for the development and operation of autonomous vehicles [6][CMC+24]. This widespread adoption is underscored by recent industry reports. According to McKinsey's "The state of AI in 2025: Agents, innovation, and transformation", 88% organizations now use AI in at least one business function, with generative AI important focus for many companies. The Artificial Intelligence Index Report indicates that, from 2019 to 2024 worldwide, the number of funded AI companies increased by 60%, meaning robust investment and innovation in the field. [7][MYCR25] Moreover, when observing the use of AI within the companies reveal more than threefold increase in the number of employees are adopting generative AI into their works when comparing with the previous year's [8] [MFP+25]Despite the substantial advancements and pervasive integration of AI, experts in this field are increasingly addressing critical questions : Should we trust blindly in these AI-generated predictions and results? Does AI support human-centered approaches? Can we evaluate the explainability and transparency of AI? Such inquiries highlighted, the inherent need for robust methodologies and frameworks for establishing AI ethics.[9] [DSHV25]. The need is crucial to overcome the problem of opacity in AI results, referred to as "black box". A powerful technique is deep learning, which uses multi-layered neural networks to process complex data, however, the intricate nature of these networks often causes their decision-making process to be opaque, which hinders the establishment of trust in AI systems. This opaqueness becomes particularly problematic in the fields of healthcare, finance, and autonomous systems, where decisions have a significant impact on human life.

Consequently, the demand for Explainable AI has urged us to enhance the transparency and interpretability inherent in complex AI models [10] [YWW+23]. Explainable Artificial Intelligence (XAI) refers to the set of methods and techniques that aim to present AI systems' decisions and functionalities in a human-understandable manner. Specifically, in image classification tasks, the use of Explainable AI techniques has become the primary focus through auditing model behavior. Post-hoc approaches in XAI focus on methods to simplify decision making logic, that are not intrinsically understandable to human users.[12] [ZWK25] These techniques, most notably Saliency Maps and Attribution Methods, aim to focus attention on the specific pixels or spatial regions that triggered a classification such as identifying the texture of a lesion in a medical scan, spatial sequence in biopsy resolution which allows to identify the risk factor in pregnancy or the facial recognition from images or video footage, or the shape of a pedestrian in autonomous driving.

However, despite their visual appeal, there is a growing consensus that many existing XAI methods fail to meet expectations, generally due to the technical limitations or lack of human-centered design.[11] [BLL+25] A preliminary study conducted using the CIFAR-10 dataset and the LIME (Local Interpretable Model-agnostic Explanations) explainer, anomalous explanation patterns were observed. Precisely, the explainer highlighted obscured local areas, where the areas are not relevant features of the objects themselves. This anomaly influenced class predictions, suggesting that the model was not performing generalized feature extraction but memorizing specific,

irrelevant pixel-level artifacts from the training data.

Such observations address the lack of underlying memorization phenomena , where models utilize superficial correlations in the training data instead of learning robust representations. This behavior stress concerns about the faithfulness of post-hoc explanations, and their ability reflect model reasoning.

Consequently, this thesis seeks a bridge this gap by investigating the extent to which model memorization results in a misleading post-hoc explanations, using these observed anomalies as a baseline, extending this baseline with different post-hoc explanation methods to develop more faithful auditing frameworks for image classification models.

Figure 1 shows the overall structure of this thesis.

This thesis is organized as follows:

– Chapter 1: Introduction introduces the fundamental questions regarding the necessity of Explainable AI (XAI), the reliability of these systems in critical sectors, and the specific impact of model memorization on the fidelity of XAI outputs.

– Chapter 2: Literature Review and Background After a deep understanding and research on the context, 'Literature Review and Background' section proceed in detail with relevant articles related Image Classification, Explainable AI, Memorization Phenomenon.

– Chapter 3: Methodology section, provides detailed mathematical approach of explainable ai techniques, and CNN architecture that will be used in Implementation

– Chapter 4: Implementation and Structure section, provides the implementation process of the thesis which will introduced in appendix.

– Chapter 5: Evaluation and Results section evaluates the implementation results.

– Chapter 6: Conclusion and Future Work Finally, the whole project is concluded, and the future works are provided in section 'Conclusion and Future Work'.

0.2 Literature Review and Background

0.2.1 Foundations of Image Classification and Deep Learning

Image Classification

Image Classification is a categorization and labeling process by mapping high dimensional pixels data to a pre-defined set of discrete classes. For decades, researchers have developed advanced classification models due to the increasing complexity in image analysis, making it critical to improve classification accuracy and reduce labor force. CNN (Convolutional Neural networks) -based approaches have significantly advanced in the field of computer vision, especially in image classification tasks, their hierarchical feature extraction mechanism, and spatially invariant filters to efficiently learn robust features of an input image. The evolution of CNN architectures has marked a significant shift from shallow architectures to deep networks due to the evaluation of increasing complexity of datasets. In early breakthrough, AlexNet is a foundational architecture that utilises GPU(Graphical Processing Unit), and significantly improved the architecture of previous convolutional neural networks using multiple layers , dropout , ReLU activation function which mitigates the vanishing gradient problem, and data augmentation to prevent overfitting. Subsequently in GoogleNet, the inception module offers a method to introduce parallel convolutional layers with varying filter sizes, which allows the network to capture features with different scales while maintaining computational efficiency .(Noor & Ige, 2024) .VGGNet , on the other hand, focus on the principle of using convolutional layers with small 3X3 filters , demonstrating that increasing the network depth could improve feature extraction capabilities. In contrast to VGGNet's emphasis on depth , in 2015 at the ILSVRC (Image Net Large Scale Visual Recognition Challenge) , ResNet was introduced with residual connections, allowing deeper networks to be trained effectively by creating skip connections allows the features learned in the previous layers to be directly reused in the upcoming layers that facilitate the backpropagation. Recent advancements in CNN Architecture the vanishing gradient problem, diminishing feature reuse while elevate the classification error by improving the feature learning . DenseNet was introduced in 2016, follows the same methodology as ResNet,utilizing residual connections between layers . However, DenseNet establishes a dense block where it consists multiple convolution functions followed by batch normalization, ReLU activation function ,and 3x3 convolution ,maximizes the information flow ,feature reuse and preserves the feedforward nature of the network. [MNI25]. ResNeXt addresses the issue of diminishing feature reuse by capturing more efficient and diverse features . While traditional networks improve performance by increasing the depth, often at the cost of computational efficiency, ResNeXt introduces a third dimension : Cardinality. By integrating residual connections of ResNet with the parallel path methodology of Inception modules, ResNeXt manages complex interactions without a computational explosion.

pre-trained cnn models with ImageNet dataset: [MNI25]

Collectively, these innovations illustrate clear evolutionary progression in CNN design, each share common underlying principles that govern how convolutional neural networks function with visual information. Understanding these fundamental concepts is essential for further analysis on memorization phenomenon in CNN architectures.

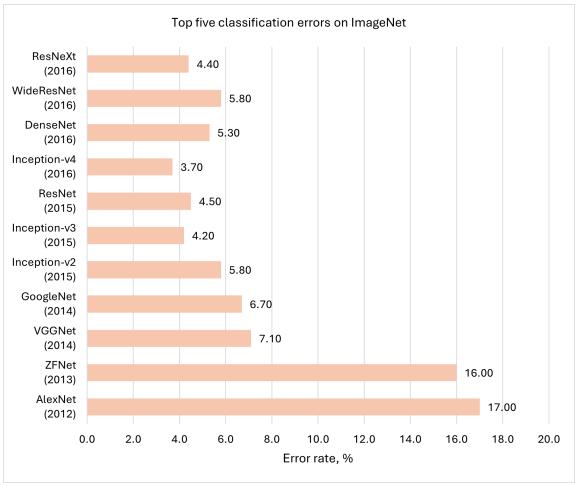


Figure 1: The classification error comparison between pre-trained models with ImageNet dataset

Brief Overview of CNN(Convolutional Neural Network) Components

The Convolutional Neural Network is a feedforward neural network designed to automatically extract hierarchical features from input data with convolutional structures .Unlike traditional feature extraction methods, CNN does not require manual feature engineering, as learning complex features hierarchically.

A key characteristic of CNNs is their hierarchical learning mechanism. Early convolutional layers capture low-level visual patterns such as edges, corners, and textures, while deeper layers can progressively integrate this low-level information into more complex, abstract representations, including object parts and semantic concepts. This hierarchical learning enables CNNs to achieve remarkable performance in various computer vision tasks, including image classification, object detection, and semantic segmentation [13] [Fel25]. Additionally, the specialized structure of CNN, which includes convolutional layers, pooling layers, and fully connected layers, facilitates parameter sharing and reduces the total number of learnable parameters, making them efficient for processing high-dimensional image data [14][BJY+18], [15][GNKN23].

The transition from traditional structure to modern structural innovations has empowered CNNs with higher dimensional capacity. While these advancements, such as non-linear activations , residual learning , more layered structure, enable the model to capture intricate details.

Despite the elevated structure, and high classification accuracy, the inherent opaqueness of these complex models often limits their interpretability, making it challenging to understand the underlying reasons for their predictions [16][IS23]. This lack of transparency becomes particularly critical in image classification tasks, where high accuracy may mask memorization effects.

Consequently, understanding of CNNs encode and utilize learned features requires the use of explainable artificial intelligence (XAI) techniques, which aim to reveal and interpret the internal representations and decision making mechanisms of deep neural networks.

Feature Extraction vs. Classification

The architecture of a Convolutional Neural Network is primarily characterized by two stages : feature extraction and classification. Understanding the distinction between these two stages is crucial for evaluating the reason behind the irrelevant pixel-level artifacts identified in image classification tasks.

The Role of Feature Extraction

Theorem 0.2.1 *The convolution of an input x with a filter w, parameterized by Q = w,b, for a single channel, is defined as:* $(x \otimes w)_{i,j} = \sum_m \sum_n x_{m,n}.w_{i-m,j-n} + b$ *where* $\otimes$ *denotes the convolution operation. For multichannel inputs, this operation is performed independently for each channel and summed to produce a single output.*

The feature extraction stage, comprising convolutional and pooling layers, performs a hierarchical decomposition of a raw data input into more abstract and semantically meaningful representations. Convolutional layers, by applying filters to the input image, generate a feature map . This feature map highlights the specific patterns and features, such as edges, corners , and textures, present in the image [16][FWN21]. Subsequently, pooling layers facilitates the reduction of dimensionality through down sampling. This reduction helps to reduce complexity by retaining the most salient features, while also enhancing the model's robustness to spatial variations and minor distortions in the input [13][Fel25].

The Role of Classification

Following the feature extraction stage, fully connected layers also known as multilayer perceptrons take the flattened output of the feature extraction to perform a global synthesis of the learned patterns. Each neuron in these layers is connected to every activation in the previous layer, performing a linear transformation followed by a non-linear activation function.

Mathematically, this operation is expressed as :

Theorem 0.2.2 $y_i = \sigma(Wx + b)$ *where x denotes the input vector, W denotes the weight matrix, b is the bias term, and* σ *is the non-linear activation function.*

This process functions as a decision-maker, integrating high dimensional feature vectors into specific semantic labels to produce the final classification. While convolutional layers focus on local spatial hierarchies, the fully connected layers interpret these features globally to determine the final probability distribution of the classes.

0.2.2 The Taxonomy of Explainable AI in Image Classification

This section establishes a comprehensive taxonomy of Explainable Artificial Intelligence and delves into mathematical foundations . The explorations focus on the categorization of XAI methods ante-hoc versus post-hoc, global versus local, and model-specific versus model-agnostic techniques, while integrating the underlying mathematical principles that govern their interpretability [Fer23, SAS25, SF23]. The essential focus is directed towards post-hoc techniques, given their substantial used in giving explanations in complex, black-box models. Understanding the post-hoc techniques ,generally fall into Attribution Methods and Model-Agnostic Local Explanations, leverage the understanding of the phenomenon of Memorization .

Following flowchart illustrates the taxonomy of explainable ai in this thesis.

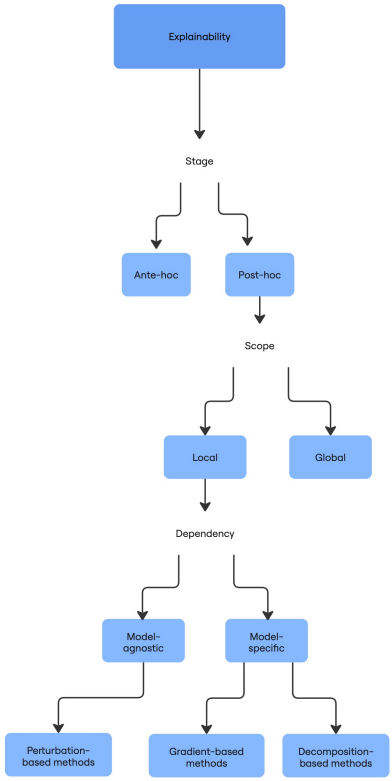


Figure 2: The flowchart of taxonomy explanation

The Categorization by Stage

XAI methods can also be described based on the stage of interpretability. This categorisation is derived at which the explanation is generated: during the model design (ante-hoc) or after the model has been trained (post-hoc).

Ante-hoc Interpretability Ante-hoc Interpretability are known as intrinsic explainability, transparent or glass-box models, which are inherently interpretable because of their uncomplicated structure.(Retzlaff et al., 2024) This simpler structure allows direct comprehension of internal decision making process . Despite their intrinsic interpretability of ante-hoc approaches, the predictive power of such models often diminishes with increasing complexity of the underlying features and data, specifically in high dimensional image processing tasks, where the straightforward linear relationships and decision trees struggle to capture complex patterns in image pixel regions.

Post-hoc Interpretability Contradictory to ante-hoc methods, post-hoc interpretability refers to the techniques which analyse and interpret the decision-making process of a trained model after it

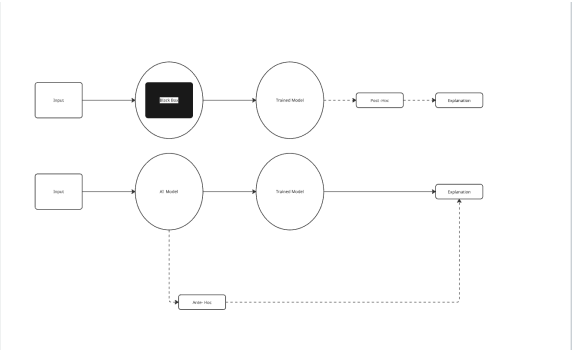


Figure 3: Different Interpretability Approach in Deep Learning Model

has made predictions, providing explanations how the model arrived at its outputs. This approach focuses on the need to simplify the decision logic of black-box models to human friendly forms.

In high-dimensional domains such as image classification, achieving high level of prediction performance is often prioritized over transparency. Additionally, the unknown relationships and intricate patterns that constrained in simpler models necessitate the use of post-hoc approaches that can reveal the hidden complexities. For this reason, post-hoc methods are indispensable for trustworthy deployment of deep learning systems.

Post-hoc techniques are commonly divided into two important categories : Model Agnostic, and Model Specific methods.

The Categorization by Scope

Scope un anlami

The categorisation on the basis of Scope mainly involves in two directions: global explanations, and local explanations.

Global Explainability Global Explainability aims to provide global understanding of a model's overall logic, revealing the general relationships between features and the model's predictions overall the entire dataset (Agrawal & Sharma, 2024). An explanation is considered as global if the explanator othen than why of local explanations, provides also how a decision is made.(Schwalbe & Finzel, 2021)

(Feature Importance, Partial Dependence Plots, and Surrogate Models are common techniques employed for global interpretability, offering insights into the average behavior of a model across its entire input space (Černevičienė & Kabašinskas, 2024; Höhl et al., 2024). This approach allows for comprehension of the model's general tendencies and the influence of features on its global output (Khan et al., 2024).)

Local Explainability Conversely, Local Explainability focuses on the reasoning behind a specific prediction for a single data instance, addressing how individual features contribute to the prediction for a particular input. This distinction is crucial for tailoring XAI methods to specific user needs and tasks. Specifically in image classification tasks, understanding why a model classifies a particular image , requires localised explanations that highlight specific regions or features within that individual image contributes to the decision .

Local Post-hoc Interpretation Techniques

Model-agnostic techniques Regardless of the architecture of the model, these methods aim to explain the decision making process of the model flexible and applicable across diverse machine learning models.(Devireddy, 2025; Ramachandram et al., 2025)This independence enables a universal framework, by simultaneously analysing the inputs and outputs without gaining model insights such as weights or parameters. However, such methods typically approximate the model locally or globally, which may limit their faithfulness.

Perturbation-based methods Perturbation-based methods operate by systematically perturbing parts of the input image and observing the corresponding changes of perturbed parts in the model's predictions.This approach is model-agnostic, as it does not rely on the internal architecture of the model, and can therefore be applied across wide range of models . As they rely on observing changes in the model's output in response to input perturbations, can be performed in black-box settings. Since the information in images is typically highly correlated across neighbouring pixels, perturbing individual pixels is unlikely to influence on the prediction.(Pihlgren & Främling, 2024). Instead, the input image is commonly perturbed at the level of segmented regions.Each segmented regions is then sampled with respect on the predefined sampling strategy that determines which regions should be perturbed or occluded. Based on this strategy, multiple perturbed instances are generated , while the remaining regions are kept unchanged. Each perturbed segment is then passed through the trained model, and resulting in a measure of the prediction change with respect to the original output. Based on these prediction differences, an attribution score is assigned to each region, reflecting its relative importance in the model's prediction alterations. Finally, these scores are aggregated to generate a saliency map, a visual explanation that highlights influential regions of the image.

Representative perturbation-based methods include LIME, Kernel SHAP which all utilize perturbation by leveraging individual features in a model.

Model specific techniques Conversely, model specific techniques leverage the internal structure of the AI model or algorithm, to provide more insightful and effective explanations such as learned parameters or gradients . These methods often tailored to specific machine learning models, especially deep learning models. While this model-specific explanation techniques can yield more precise and architecture dependent explanations , their applicability into practice if often limited, this incapability restricts their acceptance into mainstream research of XAI methods.(Hassija et al., 2023). Additionally, this approach is inherently applicable in Convolutional Neural Networks, where convolutional layers , feature maps, and gradients are directly utilise to highlight specific regions of the input image contribute most to a prediction.

Gradient-based methods Gradient-based methods, are widely used in convolutional neural networks, assume that the parameters of the model are accessible.These techniques employ forward and backward pass to estimate the influence of each input pixel respect with the original output. During the forward-pass ,the input image is passed through a deep neural network to generate a prediction score for a specific class. In the subsequent backward step, instead of adjusting weights , the gradient of the output score using back-propagation with respect to the input pixels is computed. This gradient then indicates the sensitivity of prediction to slight changes in each pixel, thereby highlighting which features the model is important related to model's prediction.

Saliency Maps, Grad-Cam, and Integrated Gradients are commonly used techniques under the gradient-based methods .

Decomposition-based methods Decomposition-based techniques decompose the model output layer by layer into different features and showing the contribution of each input feature relevance towards the final prediction. While gradient-based approaches that measure the sensitivity of the output to small changes in input features, decomposition-methods focus on highly localised features which results in more precise attributions.

A prominent example is Layer-wise Relevance Propagation (LRP), which systematically redistributes the prediction score to the previous layer, based on specific LRP rules to assign a relevance score to each input feature.

0.2.3 The Phenomenon of Memorization

Memorization is a phenomenon where machine learning models, particularly deep neural networks, retain information about specific training examples instead of learning patterns that generalize well to unseen data. This concept is crucial in understanding the behavior of the model, especially in the context of overfitting, where a model achieves high accuracy on training data but performs poorly on the test set. The high capacity and complex parameter space of deep neural networks enable these models to store detailed information from individual training samples, which causes poor generalisation and makes them susceptible to capturing noise rather than robust, transferable features [Raz23].

Researchers have further refined the concept of memorisation, often distinguishing mainly between Memorization Learning , which refers to the process of learning specific details or particular features of examples, and Pattern Learning which focuses in on extracting common patterns or generalised features in the data [WZZ$^+$24]. Understanding the distinction is crucial for diagnosing model's behaviour, and developing explanaible AI systems that can reveal whether a model' decision is driven by the memorisation or meaningful, generalisable patterns.

The following subsections expand on these ideas: Conceptual Deepening, provides the theoretical and mathematical background of Memorization giving ; Generalisation Tradeoff which examines the distinction between Generalisation vs. Memorisation;Memorisation in Convolutional Neural Networks examines the reason why CNNs are particularly prone to memorisation; and Mitigation Techniques for Memorization survey the methods to reduce memorization in deep learning models.

Conceptual Deepening of Memorization

Memorization refers to the ability of model's parameters to store and recall specific training examples rather than learning underlying data distributions. The traditional view of generalization posits that deep neural networks due to their over-parameterized, complex structure results in overfitting the training set yields poor generalization to validation set and test sets [GSS15]. However , empirical evidence shows that deep neural networks often achieve excellent generalisation performance , 'well' accuracy score with sufficient capacity [AJB$^+$17]. [ZBH$^+$21] highlight a remarkable property of deep neural networks: despite achieving zero training error, implies overfitting, these models can still maintain low test error. The real question is how these models achieves this state-of-art conditions? [ZBH$^+$21]furthers this question by varying experimental conditions on neural networks ,examining how generalization performance changes under data corruptions and label corruptions on CIFAR-10. Importantly ,even when the corruption destroys the relationship between images and labels, networks learning capability slows down yet neural networks are still able to fit the training data perfectly. - Burda generalization error formulayi ver Furthermore, this experiment highlights the rising testing error under the data corruption, emphasizing that there is a strong memorization capability. These oservations could not able to explain with a statistical learning theory where we imply different complexity measures, controlling generalisation error. These include VC Dimension [Vap99], which quantifies the complexity of a hypothesis space independently of the underlying data distribution; and its pseudo-dimension. Rademacher complexity [SBD15], refines VC Dimension including distribution dependent complexity that naturally apply to real-valued and multi-class function classes. Finally, Uniform stability measures the sensitivity of a learning algorithm to small perturbations in the training set. The observed distinction between generalization and memorization underscores that achieving low training error do not necessarily imply that meaningful patterns have been learned. Classical complexity measures alone appear insufficient to fully

explain this paradox. To explain memorization beyond classical complexity measures, it is essential to examine the nature of the information being retained. Specifically, recent literature distinguishes between two fundamental mechanisms: label memorization and feature memorization.

Label versus Feature Memorization Label Memorization refers to the phenomenon where models memorize the labels in training datasets without learning the relevant patterns that the label can be predicted . This propensity for label memorization, cannot be explained by the classical generalization formula , which is the sum of the empirical error in training set and the generalization gap controlled by the model complexity parameter and the empirical error. [ZBH+21], conducted experiments demonstrating the label memorization phenomenon on CIFAR-10. Their results show that, models trained on datasets with random labels achieve almost zero-training error, while experiencing severe generalization degradation. Specifically, they observed that as label corruption increases 0% to 100%, the convergence increases significantly, yet models continue to memorize the corrupted labels, resulting in high generalization error (since the training error is 0). In contrast, feature memorization involves the model learning and retaining spurious features that are uncorrelated with the true underlying data distribution.[BM23] [YGDM23] This type of memorization, stated in (Yu et al., 2023) , is separable based on the model's output early in training. In the experiment, a colored MNSIT used as an example of spurious correlations. Each digit in MNSIT datasets is a class , where the majority of digits have a particular color, and the remaining digits are in other colors. The model, therefore, learn to associate the color with the core feature instead of the digit's shape, leading to poor generalization on correlations where the spurious correlation between color and digit is broken [YDM+25]. While feature memorization describes the model's propensity for learning uncorrelated patters, a closely related phenomenon known as Shortcut Learning explains the mechanism behind this behaviour.

Shortcut Learning and its Implications Shortcut Learning in neural networks, occurs when a model achieves high performance on a standard benchmark, not by comprehending the underlying data correlations between input and output, but using shortcut features or exploiting trivial associations that fail in generalization performance [GJM+20]. As an example , a model trained to distinguish between wolves and huskies might learn to associate snow in the background with wolves, rather than focusing on the animals' distinct facial features [YDM+25]. Another example is provided in (Geirhos et al., 2020) an image classification task where the model is trained to identify stars and moons , instead of learning the actual shape of the objects , it exploit the location of the images .This indicates that the model has learned a "shortcut"- a decision rule that performs well on training distribution but fails in generalization tests.

This phenomenon has serious implications in applications requiring robustness and fairness . In the article [BG18] conducted an experiment over the 3 different parliamentary benchmarks to illustrate the social amplification on neural networks. Given benchmarks were IJB-A US Government Benchmark published by NIST (National Institute of Standards and Technology) in 2015 and Adience is a widely-used face image dataset originally developed for gender and age classification; yet these models overrepresented lighter males, underrepresented darker females, thereby underrepresented of darker individuals in general. PPB (Pilot Parliaments Benchmark) was therefore focused on achieving better intersectional representation in balanced datasets. To provide a comprehensive evaluation of these biases in real world applications; examined three commercial gender classification systems: Microsoft, IBM, and Face++. These classifiers were selected because they represented the state-of-the-art in facial analysis technology and were widely deployed in commercial products

Dataset Properties Comparison between Benchmarks									
Dataset	**n**	**Female**	**Male**	**Darker**	**Lighter**	**DF**	**DM**	**LF**	**LM**
PPB	1270	44.6%	55.4%	46.4%	53.6%	21.3%	25.0%	23.3%	30.3 %
IJB-A	500	24.6%	75.4%	20.4%	79.6%	4.4%	16.0%	20.2 %	59.4 %
Adience	2284	52.0%	48.0%	13.8%	86.2%	7.4%	6.4%	44.6%	41.6 %

during that period of time. The result demonstrates that even with a balanced representation of protected characteristics, models may still exhibit bias amplification in darker-skinned people. Especially the highest error rates were observed for darker-skinned females ranging from 23.8% for Microsoft to 36.0% for Face++ and 33.1% for IBM [KAR20]. The underlying cause is that the models tend to learn the shortcut way , relying on "facial geometries" and "gender norms" specific to the majority group of population in the training set, failing to generalize to minority subgroups.

When a model's decision boundary is defined by memorized shortcuts rather than learning robust semantic representations, it becomes vulnerable to critical security issues. In the following section , memorization based fragility is targeted through white-box adversarial attacks, specifically focusing on the mechanisms of FGSM and PGD.

White-Box Attacks in DNN and Its Correlation with Memorization

To evaluate the security implications of memorization, this section focuses on White-Box attacks. Unlike Black-Box scenarios where the attackers could not have any knowledge of the internal architecture, parameters or training data of the model, White-Box scenarios allows studying memorization by examining model's vulnerabilities, specifically "jagged" points where the model can be systematically exploited. To understand these vulnerabilities, the adversarial space should be defined. Adversarial Space according to [CF18], is a result of imperfect representation in training data where the entire input space cannot be provided . This imperfect representation of the input space provided in training data in white-box models , allows Attackers exploiting "Sensitivity" of memorized features and moving the direction that the model is "weakest". In standard training , gradient-descent is utilized as an optimization algorithm to minimize the cost function by updating model weights , attackers used the transparency of white-box models to reverse this logic. Instead of updating weights to minimize loss, they keep the weights fixed and perturb the input pixels to maximize the loss function. To illustrate this phenomenon , we will examine two primary gradient-based strategies: Fast Gradient Sign Method(FGSM) and Projected Gradient Descent (PGD).

Fast Gradient Sign Method (FGSM) The Fast Gradient Sign Method (FGSM) , introduced in [GSS15] adding subtle perturbations into the gradient of the cost function with respect to the original input to maximize the misclassification and thereby highlighting vulnerabilities in deep learning networks. Specifically, the sign of the gradient indicates the direction in which the input pixels must be altered to increase the loss function J. By moving the input in this direction by a small step , FGSM creates an adversarial image xadv that is often indistinguishable from the original image without visible changes but results in incorrect classification by the model. A key theoretical insight behind FGSM, is that neural networks are too linear to resist linear adversarial perturbations [GSS15] . Components such as ReLU and maxout networks are intentionally designed to behave

linearly to facilitate easier optimization during training. As a consequence, the cumulative effect of a small change in each input dimension results in a drastic change in the final output. FGSM exploits this linearity ; the attacker moves the input across the sign of the gradient with minimal computational effort, making it one of the most efficient adversarial attack strategies. This is empricaly demonstrated in " panda vs. gibbon" example , where a perturbation of merely $= 0.007$ is sufficient to cause misclassification. As discussed , the adversarial space emerges from the imperfect

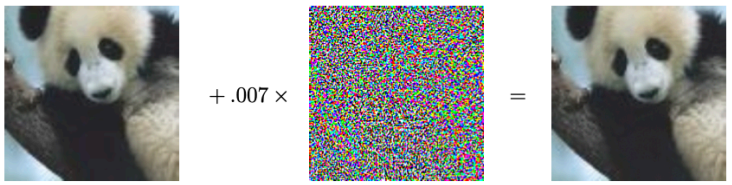


Figure 4: Adversarial Example Generated by FGSM

representation of the input space during training. Memorization amplifies this imperfections by increasing the model's susceptibility to gradient-based perturbations. When a model prioritizes memorizing specific samples over learning generalizable features, the resulting decision boundaries become highly "jagged" and fragile. These boundaries narrow the distance between training data, which attackers can systematically exploit using fast gradient sign method to shift the input into incorrect classification zone.

Projected Gradient Descent (PGD) While FGSM provides single-step attacks , PGD provides iterative approach by applying small perturbations to the input data to maximize the model's prediction error. Based on the information given in [MMS+17], the inner problem is to find a perturbation that causes the greatest increase in loss function. In prior studies , the linearization approach that is used in FGSM , a single step approach , is not enough to understand the inner problem in more detail. The iterative nature of PGD allows for a more exhaustive search for adversarial examples repeatedly applying first-order gradients to navigate the complex loss landscape. In the experiment provided in [MMS+17], states that given datasets of MNSIT and CIFAR-10 , the adjusted PGD attack consistently outperformed FGSM in terms of attack success, with a higher percentage of loss maximization with CIFAR-10 dataset. Even though MNSIT and CIFAR-10 are robust to adversarial attacks, MNSIT achieves high accuracy for wide range of powerful ∞-bound adversaries and large perturbations due to the large fraction of atypical samples contained in CIFAR-10 .

Furthermore, Memorization amplifies the effectiveness of PGD by exploiting the model's vulnerabilities in training by generating a very jagged loss landscape. While PGD is a first-order (gradient-based) adversary, if the gradients are "shattered" or "noisy" due to memorization, PGD might get stuck in poor local maximum, making the model appear robust when it actually isn't.

0.3 Methodology

This chapter outlines the mathematical foundations and methodologies underlying the investigation of memorization in deep neural networks. The methodology is structured systematically: beginning with the mathematical background of neural networks and training, formalizing the concept of memorization, an exploration of white-box attack techniques, and an examination of explainable AI (XAI) methods to reveal and interpret memorized representations in image classification models. Finally, we detail the experimental setup, audit pipeline, and evaluation metrics used to analyze the results.

0.3.1 Mathematical Background of Neural Networks and Training

A convolutional Neural Network (CNN) is a specialized kind of neural networks employs a mathematical operation called convolution. Convolution is a mathematical operation that combines two functions to produce a real valued output. In the context of CNNs, it is used to extract features hierarchically from input data, such as images, followed by non-linear activation functions to introduce non-linearity into the model. After extracting features through convolutional layers and pooling layers, the output is passed through fully connected layers to perform classification or regression tasks. The convolution operation can be mathematically expressed as follows:

$$z(t) = (x * w) \cdot (t) \tag{1}$$

where $z(t)$ is the output signal, x is the input image, w is the kernel (or filter), and t represents the spatial location in the output feature map. The convolution operation involves sliding the kernel across the input signal and computing the dot product at each position to produce the output feature map.

Forward Pass and Activation Functions

$$z_{i,j}^{(l)} = (W * X) \cdot (i,j) = \sum_m \sum_n W_{m,n}^{(l)} \cdot x_{(i+m,j+n)} \tag{2}$$

where $z_{i,j}^{(l)}$ is the pre-activation value at position (i,j) in layer l, W is the weight matrix (or filter), X is the input image, and (i,j) represents the spatial location in the feature map. The kernel W is a small matrix of weights that is sliding across the input image X to capture the local features. The output feature map $z_{i,j}^{(l)}$ is generated by summing the element-wise product of the kernel and the corresponding region of the input image. This operation allows CNNs to learn spatial hierarchies of features.

In a CNN, the forward pass is the sequential convolution operation of an input image trough series of layers to produce predicted output. At each layer l, the feature map is computed applying the convolution operation defined in Equation 2 followed by the non-linear activation function $\phi(\cdot)$. The mathematical representation of the forward pass can be expressed as follows:

$$a^{(l)} = \phi(z^{(l)}) \tag{3}$$

where $a^{(l)}$ is the activated output layer l, $z^{(l)}$ is the output of the convolution operation at layer l, and $\phi(\cdot)$ is the activation function.

Convolutional Neural Networks can utilize various activation functions to express non-linearity, such as sigmoid, tanh, and Rectified Linear Unit (ReLU), which can be mathematically defined as follows:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{4}$$

Sigmoid activation function, as shown in Equation 4 maps the input to a real value between (0,1) which is useful for binary classification problems. Even though sigmoid function is widely used, it suffers from vanishing gradient problem , which can hinder the training of deep neural networks. The vanishing gradient problem occurs when the gradient becomes almost zero for large or small values of z. The graph of the sigmoid function is shown in Figure 5

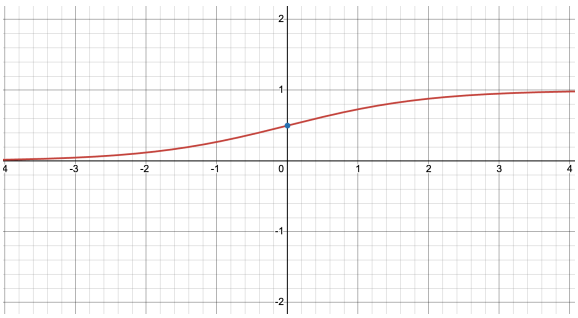


Figure 5: Graphical representation of the sigmoid activation function.

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}} \tag{5}$$

Tanh activation function, as shown in Equation 5 maps the input values to a range of (-1,1) which can be useful for certain classification tasks where the output can be negative. However, similar to sigmoid function, tanh also suffers from the vanishing gradient problems. The graph of the tanh function is shown in Figure 6 ReLU introduces non-linearity by mapping input values smaller

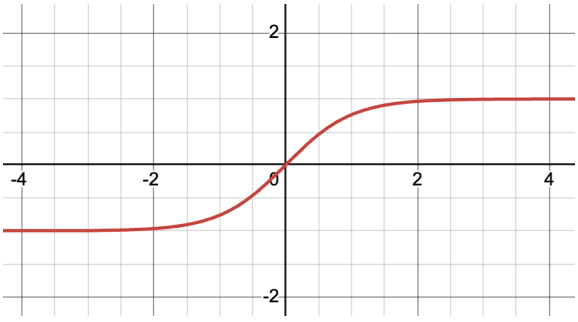


Figure 6: Graphical representation of the tanh activation function.

than zero to zero and keeping positivive values unchanged. The mathematical representation of the

ReLU function is given by Equation 6.

$$\text{ReLU}(z) = \max(0, z) \tag{6}$$

ReLU is computationally efficient and helps mitigate the vanishing gradient problem, compared to sigmoid and tanh functions. The function converges faster during training and allows for deeper networks, which is the reason why it is widely used in CNNs. However, the ReLU function can suffer from the "Dying ReLU" problem, which occurs when the input values are negative and the gradient becomes zero. In that case, the neurons will be regarded as inactivated or dead. The graph of the ReLU function is shown in Figure 7

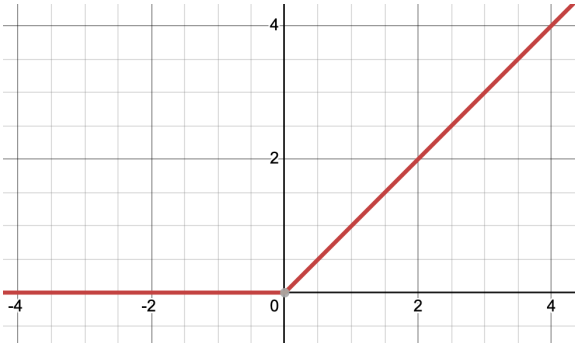


Figure 7: Graphical representation of the ReLU activation function.

For the output layer, the Softmax function maps the raw output scores of the final layer to class probabilities:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \tag{7}$$

Where $\hat{y}_k$ denotes the predicted probability corresponding to class k. z_k denotes the raw output score for class k from the final layer of the network, and the denominator sums over all classes iterating j to compute the normalization factor. The softmax function ensures that the output probabilities sum to 1, making it suitable for multi-class classification problems.

Loss Function and Optimization

Given a training set Given a training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, where x_i denotes the i-th input image and y_i the actual output of the model, the model is trained by minimizing the empirical risk using a loss function that quantifies the distance between the predicted value and the actual value :

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i) \tag{8}$$

Where θ denotes the model parameters, $f_\theta(x_i)$ is the predicted output, n denotes the number of samples and $\ell(\cdot, \cdot)$ is the loss function corresponding to the task. Especially for classification tasks, cross-entropy loss often referred to as the softmax loss is commonly used to measure the probability

distribution difference between obtained and actual output. [**?**] In multiple class datasets, such as CIFAR-10, MNSIT, ImageNet the cross-entropy loss is defined as:

$$\ell(\hat{y}, y) = -\sum_{k=1}^{K} y_k \log \hat{y}_k \tag{9}$$

Where K is the number of classes, $y_k \in \{0, 1\}$ is the ground-truth indicator for class k, and $\hat{y}_k$ is the predicted probability defined in Equation 7. The training objective is therefore to find the optimal parameters:

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta) \tag{10}$$

Backpropagation and Gradient Descent

To minimize the loss function defined in Equation 8, the model parameters are updated iteratively using gradient descent algorithms. The gradient descent algorithm can be expressed as follows:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t) \tag{11}$$

Where θ_{t+1} denotes the updated parameters at iteration $t+1$, θ_t denotes the previous parameters at iteration t, η is the learning rate, and $\nabla_\theta \mathcal{L}(\theta_t)$ is the gradient of the loss function with respect to the parameters at iteration t.

In Convolutional Neural Networks (CNNs), the gradients are computed via backpropagation, which applies the chain rule to calculate the gradients of the loss function. For a layer l,the gradient of the loss function with respect to the weights is:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial W^{(l)}} \tag{12}$$

Where $\frac{\partial \mathcal{L}}{\partial z^{(l)}}$ denotes the error of the current layer l, $\frac{\partial z^{(l)}}{\partial W^{(l)}}$ is the input according to the weights of the current layer. The error signal is propagated backward through the network, allowing for the computation of gradients for each layer.

The simplest gradient descent algorithm is Stochastic Gradient Descent

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_{\mathcal{B}}(\theta_t) \tag{13}$$

Where η is the learning rate. While SGD is effective, it applies a uniform learning rate to all parameters. Adam extends SGD by maintaining adaptive moment estimates of the gradients:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla \mathcal{L}, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla \mathcal{L})^2 \tag{14}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \tag{15}$$

Where $\hat{m}_t = \frac{m_t}{1-\beta_1^t}$ and $\hat{v}_t = \frac{v_t}{1-\beta_2^t}$ are bias-corrected estimates, β_1 and β_2 are decay rates, and ϵ is a small constant for numerical stability. Notably, the simplicity bias of SGD-based optimizers toward simpler solutions has been identified as a key factor in the emergence of spurious correlations and memorization during training [YGDM23], which is further examined in Section 3.2.

0.3.2 Mathematical Formalization of Memorization

Memorization in deep neural networks refers to the phenomenon where a model stores specific training examples rather than learning generalizable patterns from the underlying data distribution. Formally, given a training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, a model f_θ is said to memorize a training example (x_i, y_i) if its prediction is driven by instance-specific features rather than the true underlying distribution $p(y|x)$.

Generalization and Memorization

The classical generalization framework defines the generalization error as:

$$\mathcal{E}_{gen} = \mathcal{E}_{test} - \mathcal{E}_{train} \tag{16}$$

Where $\mathcal{E}_{train}$ is the empirical training error and $\mathcal{E}_{test}$ is the test error. A model that memorizes training examples achieves low $\mathcal{E}_{train}$ while exhibiting high $\mathcal{E}_{test}$, resulting in a large generalization gap. However, as demonstrated by [ZBH+21], this classical view is insufficient to fully explain memorization — deep neural networks can achieve near-zero training error even on randomly labeled data, yet still generalize well on correctly labeled data. This paradox motivates a more refined formalization.

Memorization Score

To quantify memorization at the instance level, the memorization score of a training example (x_i, y_i) is defined as the influence of that example on the model's prediction [**?**]:

$$\text{mem}(f, \mathcal{D}, i) = \Pr_{f \sim \mathcal{A}(\mathcal{D})}[f(x_i) = y_i] - \Pr_{f \sim \mathcal{A}(\mathcal{D} \setminus \{i\})}[f(x_i) = y_i] \tag{17}$$

Where $\mathcal{A}$ is the training algorithm, $\mathcal{D} \setminus \{i\}$ denotes the training set with example (x_i, y_i) removed, and the difference measures how much the model's correct prediction of x_i depends on having seen that example during training. A high memorization score indicates that the model has memorized the example rather than learned a generalizable pattern.

Label Memorization

Label memorization occurs when the model fits the training labels without learning the underlying feature distributions. Formally, given a corrupted dataset $\tilde{\mathcal{D}} = \{(x_i, \tilde{y}_i)\}_{i=1}^n$ where $\tilde{y}_i$ is a randomly assigned label, a model exhibits label memorization if:

$$\mathcal{E}_{train}(\tilde{\mathcal{D}}) \approx 0 \quad \text{while} \quad \mathcal{E}_{test}(\tilde{\mathcal{D}}) \approx \frac{K-1}{K} \tag{18}$$

Where K is the number of classes. This result, demonstrated empirically by [ZBH+21] on CIFAR-10, shows that the model achieves near-zero training error on randomly labeled data while test error approaches the random chance baseline of $\frac{K-1}{K}$.

Feature Memorization

Feature memorization refers to the model retaining spurious features from specific training instances rather than abstracting meaningful representations [BM23, YGDM23]. Unlike label memorization, which concerns the retention of incorrect class assignments, feature memorization reflects the model's tendency to rely on instance-level surface features uncorrelated with the true underlying data distribution. Formally, let $x_i = v_c + v_s + \xi_i$ where v_c is the core feature, v_s is the spurious feature, and ξ_i is instance-level noise. A model exhibits feature memorization when:

$$|f(v_s; W_T, z_T)| \gg |f(v_c; W_T, z_T)| \tag{19}$$

That is, the model's output is predominantly determined by the spurious feature v_s rather than the core feature v_c [YGDM23].

0.3.3 Mathematics of White-Box Attacks (Adversarial Optimization)

White-box attacks assume full access to the model's architecture, parameters, and gradients. Unlike black-box scenarios where the attacker has no knowledge of the internal structure, white-box attacks exploit the transparency of the model to systematically craft adversarial examples. Formally, given a model f_θ, an input image x with true label y, and a loss function $\mathcal{L}$, the adversarial attack objective is to find a perturbation δ such that:

$$x_{adv} = x + \delta, \quad \text{subject to} \quad \|\delta\|_p \leq \epsilon \tag{20}$$

Where ϵ controls the magnitude of the perturbation and $\|\cdot\|_p$ denotes the ℓ_p norm. The attacker aims to maximize the loss while keeping the perturbation imperceptible:

$$\delta^* = \arg\max_{\|\delta\|_p \leq \epsilon} \mathcal{L}(f_\theta(x + \delta), y) \tag{21}$$

Fast Gradient Sign Method (FGSM)

The Fast Gradient Sign Method (FGSM), introduced by Goodfellow et al. (2015), generates adversarial examples by computing a single-step perturbation along the sign of the gradient of the loss with respect to the input:

$$x_{adv} = x + \epsilon \cdot \text{sign}\left(\nabla_x \mathcal{L}(f_\theta(x), y)\right) \tag{22}$$

Where $\nabla_x \mathcal{L}(f_\theta(x), y)$ is the gradient of the loss with respect to the input image x, and $\text{sign}(\cdot)$ extracts the sign of each gradient component. FGSM exploits the linearity of deep neural networks — components such as ReLU, defined in Equation 6, are intentionally designed to behave linearly, causing small perturbations applied consistently across input dimensions to produce drastic changes in the model output (Goodfellow et al., 2015). The connection between memorization and FGSM vulnerability is examined in Section 4.

Projected Gradient Descent (PGD)

Projected Gradient Descent (PGD), proposed by Madry et al. (2018), extends FGSM by applying multiple gradient steps with projection back onto the ϵ-ball at each iteration:

$$x_{adv}^{(t+1)} = \Pi_{x+\mathcal{S}} \left(x_{adv}^{(t)} + \alpha \cdot \text{sign} \left(\nabla_x \mathcal{L}(f_\theta(x_{adv}^{(t)}), y) \right) \right) \tag{23}$$

Where α is the step size, t denotes the iteration number, $\mathcal{S} = \{\delta : \|\delta\|_\infty \leq \epsilon\}$ is the allowed perturbation set, and $\Pi_{x+\mathcal{S}}(\cdot)$ denotes the projection operator that ensures the adversarial example remains within the ϵ-ball around the original input x. PGD is considered a stronger attack than FGSM as it iteratively refines the perturbation, making it more effective at finding adversarial examples that exploit memorized features in the model.

0.3.4 XAI and Attribution Mathematics

Explainable Artificial Intelligence (XAI) refers to a set of methods and techniques that make the predictions of machine learning models interpretable and transparent. In the context of this thesis, XAI methods are employed to reveal and interpret memorized features in CNNs trained on image classification tasks. Formally, given a model f_θ and an input image x, an attribution method computes a saliency map $\mathcal{A}(x) \in \mathbb{R}^{H \times W}$ that assigns an importance score to each pixel, indicating its contribution to the model's prediction.

Gradient-weighted Class Activation Mapping (GradCAM)

GradCAM (Selvaraju et al., 2017) computes the importance of each feature map in the final convolutional layer with respect to a target class c. The weights α_k^c for feature map k are obtained by global average pooling of the gradients:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial \hat{y}^c}{\partial A_{ij}^k} \tag{24}$$

where Z is the number of pixels in the feature map, $\hat{y}^c$ is the predicted score for class c, and A_{ij}^k is the activation of feature map k at spatial location (i, j). The GradCAM saliency map is then computed as:

$$\mathcal{L}_{GradCAM}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right) \tag{25}$$

The ReLU operation retains only the features that have a positive influence on the target class c. GradCAM produces a coarse localization map that highlights the regions of the input image most relevant to the model's prediction, making it particularly suitable for revealing memorized spatial features in CNNs.

Integrated Gradients

Integrated Gradients (Sundararajan et al., 2017) computes attributions by integrating the gradients of the model's output with respect to the input along a straight path from a baseline x' (typically a black image) to the actual input x:

$$\mathcal{IG}_i(x) = (x_i - x_i') \times \int_{\alpha=0}^{1} \frac{\partial f_\theta(x' + \alpha(x - x'))}{\partial x_i} d\alpha \tag{26}$$

where x_i is the i-th input feature, x_i' is the corresponding baseline feature, and α interpolates between the baseline and the input. In practice, the integral is approximated using a Riemann sum with m steps:

$$\mathcal{IG}_i(x) \approx (x_i - x_i') \times \sum_{k=1}^{m} \frac{\partial f_\theta\left(x' + \frac{k}{m}(x - x')\right)}{\partial x_i} \times \frac{1}{m} \tag{27}$$

Integrated Gradients satisfies the completeness axiom, meaning that the sum of attributions equals the difference between the model's output at the input and the baseline:

$$\sum_i \mathcal{IG}_i(x) = f_\theta(x) - f_\theta(x') \tag{28}$$

SHAP (SHapley Additive exPlanations)

SHAP (Lundberg & Lee, 2017) is grounded in cooperative game theory and computes the contribution of each feature to the model's prediction based on Shapley values. For a model f_θ and input x, the SHAP value for feature i is defined as:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \left[f_\theta(S \cup \{i\}) - f_\theta(S)\right] \tag{29}$$

where F is the set of all features, S is a subset of features excluding i, and $f_\theta(S)$ denotes the model's prediction using only the features in S. The Shapley value ϕ_i represents the average marginal contribution of feature i across all possible feature subsets, ensuring a fair and consistent attribution.

LIME (Local Interpretable Model-agnostic Explanations)

LIME (Ribeiro et al., 2016) approximates the behaviour of a complex model locally around a specific input x by fitting an interpretable surrogate model $g \in G$:

$$\xi(x) = \arg\min_{g \in G} \mathcal{L}(f_\theta, g, \pi_x) + \Omega(g) \tag{30}$$

where $\mathcal{L}(f_\theta, g, \pi_x)$ measures how well the surrogate model g approximates f_θ in the neighborhood of x defined by the proximity measure π_x, and $\Omega(g)$ is a complexity penalty that ensures the surrogate model remains interpretable. For image classification, LIME generates perturbed versions of the input by masking superpixels and observes the change in the model's prediction to identify the most influential regions.

Connecting XAI to Memorization

The attribution methods described above serve as diagnostic tools for revealing memorization in CNNs. A model that has memorized specific training instances will produce saliency maps that highlight instance-specific spurious features rather than semantically meaningful regions. Formally, for a memorized example (x_i, y_i) with high memorization score $\text{mem}(f, \mathcal{D}, i)$ as defined in Equation 17, the attribution map $\mathcal{A}(x_i)$ is expected to assign high importance to spurious features v_s rather than core features v_c:

$$\sum_{j \in v_s} \mathcal{A}_j(x_i) \gg \sum_{j \in v_c} \mathcal{A}_j(x_i) \tag{31}$$

This formal connection between memorization and XAI attributions constitutes the theoretical foundation of the audit pipeline proposed in Section 3.5.

0.3.5 Experimental Setup and Audit Pipeline

This section describes the experimental framework designed to reveal and interpret memorization in CNNs applied to image classification. The audit pipeline integrates model training, memorization analysis, adversarial evaluation, and explainability methods into a unified workflow.

Dataset

The CIFAR-10 dataset (Krizhevsky et al., 2009) is used as the primary benchmark for the experimental evaluation. CIFAR-10 consists of 60,000 color images of size 32×32 pixels, distributed across 10 classes with 6,000 images per class. The dataset is partitioned into 50,000 training images and 10,000 test images. To evaluate the model's behavior on out-of-distribution inputs, a set of custom images collected by the author is additionally used as a supplementary test set.

The input images are preprocessed as follows. Each image $x \in \mathbb{R}^{32 \times 32 \times 3}$ is normalized using the channel-wise mean $\mu = (0.4914, 0.4822, 0.4465)$ and standard deviation $\sigma = (0.2023, 0.1994, 0.2010)$ computed over the training set:

$$\hat{x} = \frac{x - \mu}{\sigma} \tag{32}$$

Model Architecture

A Convolutional Neural Network is trained from scratch on CIFAR-10 as the primary model for memorization analysis. The architecture consists of three convolutional blocks, each containing a convolutional layer, batch normalization, ReLU activation, and max pooling, followed by two fully connected layers for classification. The model is trained using the Adam optimizer defined in Equations 14 and 15, with a learning rate of $\eta = 0.001$, batch size of 64, and cross-entropy loss defined in Equation 9.

Memorization Analysis

To identify memorized examples, the memorization score defined in Equation 17 is computed for each training example. Examples with a memorization score above a threshold τ are classified as memorized:

$$\mathcal{M} = \{(x_i, y_i) \in \mathcal{D} : \text{mem}(f, \mathcal{D}, i) \geq \tau\} \tag{33}$$

where $\mathcal{M}$ denotes the set of memorized examples. The threshold τ is determined empirically based on the distribution of memorization scores across the training set.

Adversarial Evaluation

To assess the security implications of memorization, adversarial examples are generated using FGSM and PGD as defined in Equations 22 and 23 respectively. The perturbation magnitude is set to $\epsilon = 0.03$ for both attacks, with PGD using a step size of $\alpha = 0.007$ and $T = 40$ iterations. Adversarial examples are generated for both memorized examples $\mathcal{M}$ and non-memorized examples to compare their relative vulnerability.

Audit Pipeline

The complete audit pipeline consists of the following sequential stages:

1. **Training:** Train the CNN on CIFAR-10 using the Adam optimizer and cross-entropy loss.
2. **Memorization Analysis:** Compute memorization scores for all training examples and identify the memorized set $\mathcal{M}$.
3. **Adversarial Evaluation:** Generate adversarial examples using FGSM and PGD for both memorized and non-memorized examples.
4. **XAI Attribution:** Apply GradCAM, Integrated Gradients, SHAP, and LIME to memorized examples and their adversarial counterparts.
5. **Analysis:** Compare attribution maps of memorized versus non-memorized examples to reveal instance-specific spurious features.

0.3.6 Evaluation Metrics and Analysis

The experimental results are evaluated using the following metrics:

Classification Performance

The model's classification performance is measured using top-1 accuracy on the CIFAR-10 test set:

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}[\hat{y}_i = y_i] \tag{34}$$

where $\mathbf{1}[\cdot]$ is the indicator function and n is the number of test examples.

Adversarial Robustness

The model's robustness to adversarial attacks is measured by the drop in accuracy under FGSM and PGD perturbations:

$$\Delta_{adv} = \text{Accuracy}(f_\theta, \mathcal{D}_{test}) - \text{Accuracy}(f_\theta, \mathcal{D}_{adv}) \tag{35}$$

where $\mathcal{D}_{adv}$ denotes the adversarially perturbed test set. A larger Δ_{adv} indicates lower robustness and higher vulnerability to adversarial attacks.

Memorization Rate

The memorization rate is defined as the proportion of training examples classified as memorized:

$$\text{MR} = \frac{|\mathcal{M}|}{|\mathcal{D}|} \tag{36}$$

where $|\mathcal{M}|$ is the number of memorized examples and $|\mathcal{D}|$ is the total number of training examples.

Attribution Similarity

To quantify the difference between attribution maps of memorized and non-memorized examples, the structural similarity index (SSIM) is used:

$$\text{SSIM}(\mathcal{A}_1, \mathcal{A}_2) = \frac{(2\mu_1\mu_2 + c_1)(2\sigma_{12} + c_2)}{(\mu_1^2 + \mu_2^2 + c_1)(\sigma_1^2 + \sigma_2^2 + c_2)} \tag{37}$$

where μ_1, μ_2 are the mean values of the attribution maps, σ_1^2, σ_2^2 are their variances, σ_{12} is their covariance, and c_1, c_2 are stability constants. A low SSIM score between memorized and non-memorized attribution maps indicates that the model relies on fundamentally different features for these two groups of examples.

0.4 Implementation and Structural Design

/** In which tools and frameworks are the experiments implemented? **/ /** lsitings -> cod snippets (pseudocode) tam kodu -> appendix e tasi syntax highlighting, line numbers, vs. giving explanation above the code snippet, and then the code snippet itself in the appendix.

Development Environment

The implementation of the experiments was developed in Python 3.11, utilizing the PyTorch library for deep learning model construction and training. The experiments were conducted in a Jupyter Notebook environment, and the GPU(General Purpose Unit) acceleration was leveraged using Apple M4 hardware. The following libraries were used throughout the implementation:

- **PyTorch 2.10**: For building and training the convolutional neural network models.
- **Torchvision 0.25**: For loading and preprocessing the CIFAR-10 dataset.
- **pytorch-grad-cam** — GradCAM implementation for explainability analysis
- **Captum 0.8** — model interpretability library for Integrated Gradients.
- **NumPy 1.26** — numerical computation and array manipulation
- **Matplotlib 3.10** — visualization of training metrics and saliency maps
- **Scikit-learn 1.8** — evaluation metrics and statistical analysis
- **ipykernel 7.2.0** — interactive development and

The MPS backend of PyTorch was utilized to accelerate training on Apple M4 hardware, significantly reducing training time compared to CPU-only execution. Device configuration was handled as follows:

Data Pipeline

Model Architecture

Memorization Analysis

XAI implementation

Adversarial Attacks

0.5 Evaluation and Results

0.6 Conclusion and Future Work

0.7 Appendix

Bibliography

[AAES+23] Sajid Ali, Tamer Abuhmed, Shaker El-Sappagh, Khan Muhammad, Jose M. Alonso-Moral, Roberto Confalonieri, Riccardo Guidotti, Javier Del Ser, Natalia Díaz-Rodríguez, and Francisco Herrera. Explainable artificial intelligence (xai): What we know and what is left to attain trustworthy artificial intelligence. *Information Fusion*, 99:101805, 2023.

[AJB+17] Devansh Arpit, Stanisław Jastrz bski, Nicolas Ballas, David Krueger, Emmanuel Bengio, Maxinder S Kanwal, Tegan Maharaj, Asja Fischer, Aaron Courville, Yoshua Bengio, and Simon Lacoste-Julien. A closer look at memorization in deep networks. *Homo Politicus (Academy of Humanities and Economics in Lodz)*, 70:233–242, 08 2017.

[BG18] Joy Buolamwini and Timnit Gebru. Gender shades: Intersectional accuracy disparities in commercial gender classification. pages 77–91, 01 2018.

[BJY+18] Alsallakh Bilal, Amin Jourabloo, Mao Ye, Xiaoming Liu, and Liu Ren. Do convolutional neural networks learn class hierarchy? *IEEE Transactions on Visualization and Computer Graphics*, 24(1):152–162, January 2018.

[BLL+25] Clara Bove, Thibault Laugel, Marie-Jeanne Lesot, Charles Tijus, and Marcin Detyniecki. Why do explanations fail? a typology and discussion on failures in xai, 2025.

[BM23] Simone Bombari and Marco Mondelli. How spurious features are memorized: Precise analysis for random and ntk features. *arXiv (Cornell University)*, 05 2023.

[CF18] Chio Clarence and David Freeman. *Adversarial Machine Learning*. O'Reilly Media, 02 2018.

[CMC+24] Can Cui, Yunsheng Ma, Xu Cao, Wenqian Ye, and Ziran Wang. Drive as you speak: Enabling human-like interaction with large language models in autonomous vehicles. In *2024 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, pages 902–909, 2024.

[DSHV25] Chameera De Silva, Thilina Halloluwa, and Dhaval Vyas. A multi-layered research framework for human-centered ai: Defining the path to explainability and trust. arXiv preprint arXiv:2504.13926, 2025. Submitted on 14 Apr 2025, last revised 26 Apr 2025.

[Fel25] Thomas Fel. Sparks of explainability: Recent advancements in explaining large vision models, 2025.

[Fer23] Julien Ferry. *Addressing Interpretability, Fairness & Privacy in Machine Learning through Combinatorial Optimization Methods*. Phd thesis, Université Paul Sabatier – Toulouse III, Toulouse, France, 2023. NNT: 2023TOU30199.

[FWN21] Ahmad Rafiansyah Fauzan, Mohammad Iwan Wahyuddin, and Sari Ningsih. Pleural effusion classification based on chest x-ray images using convolutional neural network. *Jurnal Ilmu Komputer dan Informasi*, 2021.

[GJM+20] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard S. Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 11 2020.

[GNKN23] N. Ghaffar Nia, E. Kaplanoglu, and A. Nasab. Evaluation of artificial intelligence techniques in disease diagnosis and prediction. *Discov Artif Intell*, 3:5, 2023.

[GSS15] Ian Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv (Cornell University)*, 03 2015.

[IS23] Rami Ibrahim and M. Omair Shafiq. Explainable convolutional neural networks: A taxonomy, review, and future directions. *ACM Comput. Surv.*, 55(10), February 2023.

[JMH24] R. Narendra Jatna, Reda Manthovani, and Hasbullah Hasbullah. The role of disruptive artificial intelligence technology in combating crime in indonesia. *Beijing Law Review*, 15(3), 2024.

[KAR20] Anoop Krishnan, Ali Almadan, and Ajita Rattani. Understanding fairness of gender classification algorithms across gender-race groups. pages 1028–1035, 12 2020.

[MFP+25] Nestor Maslej, Loredana Fattorini, Raymond Perrault, Yolanda Gil, Vanessa Parli, Njenga Kariuki, Emily Capstick, Anka Reuel, Erik Brynjolfsson, John Etchemendy, Katrina Ligett, Terah Lyons, James Manyika, Juan Carlos Niebles, Yoav Shoham, Russell Wald, Toby Walsh, Armin Hamrah, Lapo Santarlasci, Julia Betts Lotufo, Alexandra Rome, Andrew Shi, and Sukrut Oak. Artificial intelligence index report 2025. arXiv preprint arXiv:2504.07139, 2025. Submitted on 8 Apr 2025, last revised 20 Nov 2025.

[MMS+17] Aleksander Mądry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv (Cornell University)*, 06 2017.

[MNI25] Mohd Halim Mohd Noor and Ayokunle Olalekan Ige. A survey on state-of-the-art deep learning applications and challenges. *Engineering Applications of Artificial Intelligence*, 159:111225, 2025. arXiv preprint arXiv:2403.17561v9, revised 25 Jul 2025.

[MYCR25] Hannah Mayer, Lareina Yee, Michael Chui, and Roger Roberts. Superagency in the workplace: Empowering people to unlock ai's full potential. McKinsey & Company Report, January 28 2025. Accessed 2025.

[OUM24] Lawrence Damilare Oyeniyi, Chinonye Esther Ugochukwu, and Noluthando Zamanjomane Mhlongo. Implementing ai in banking customer service: A review of current trends and future applications. *International Journal of Science and Research Archive*, 11(2):1492–1509, 2024.

[PF24] Y. Pan and A. Foroughi. Evaluation of ai tools for healthcare networks at the cloud-edge interaction to diagnose autism in educational environments. *Journal of Cloud Computing*, 13:39, 2024.

[Raz23] Ali Raza. Secure and privacy-preserving federated learning with explainable artificial intelligence for smart healthcare system, 08 2023.

[SAS25] Qiyang Sun, Alican Akman, and Björn W. Schuller. Explainable artificial intelligence for medical applications: A review. *ACM Trans. Comput. Healthcare*, 6(2), February 2025.

[SBD15] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory To Algorithms*. 01 2015.

[SF23] Gesina Schwalbe and Bettina Finzel. A comprehensive taxonomy for explainable artificial intelligence: a systematic survey of surveys on methods and concepts. *Data Mining and Knowledge Discovery*, 38(5):3043–3101, January 2023.

[SHZ+24] Sahat Maruli Tua Situmeang, Rois Harliyanto, Pingky Dezar Zulkarnain, Umar Mahdi, and Taufan Nugroho. The role of artificial intelligence in criminal justice. *Global*, 2(8), 2024.

[Vap99] Vladimir Vapnik. An overview of statistical learning theory. *IEEE Transactions on Neural Networks*, 10:988–999, 01 1999.

[WZZ+24] Jiaheng Wei, Yanjun Zhang, Leo Yu Zhang, Ming Ding, Chao Chen, Kok-Leong Ong, Jun Zhang, and Yang Xiang. Memorization in deep learning: A survey. *arXiv (Cornell University)*, 06 2024.

[YDM+25] Chenyu You, Haocheng Dai, Yifei Min, Jasjeet S. Sekhon, Sarang Joshi, and James S. Duncan. Uncovering memorization effect in the presence of spurious correlations. *Nature Communications*, 16:5424–5424, 07 2025.

[YGDM23] Yang Yu, Eric Gan, Gintare Karolina Dziugaite, and Baharan Mirzasoleiman. Identifying spurious biases early in training through the lens of simplicity bias. *arXiv (Cornell University)*, 05 2023.

[YWW+23] Wenli Yang, Yuchen Wei, Hanyu Wei, Yanyu Chen, Guangtai Huang, Xiang Li, Renjie Li, Naimeng Yao, Xinyi Wang, Xiaodong Gu, Muhammad Bilal Amin, and Byeong Kang. Survey on explainable ai: From approaches, limitations and applications aspects. *Human-Centric Intelligent Systems*, 3:161 – 188, 2023.

[ZBH+21] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning (still) requires rethinking generalization. *Communications of the ACM*, 64:107–115, 02 2021.

[ZWK25] Patrick Zschech, Sven Weinzierl, and Mathias Kraus. Inherently interpretable machine learning: A contrasting paradigm to post-hoc explainable ai. *Business and Information Systems Engineering*, sep 2025.